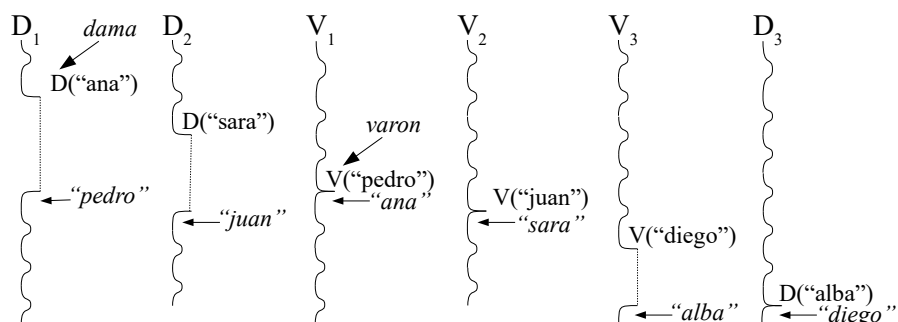


Múltiples threads representan damas y varones que buscan una pareja para bailar en una discoteca invocando las funciones *dama* y *varon* respectivamente. Ambas funciones reciben como parámetro el nombre de quien invoca la función y deben retornar el nombre de su pareja de baile. Si al invocar *dama* o *varon* no hay personas del sexo opuesto en espera, la invocación debe esperar. Si no, se elige como pareja la persona del sexo opuesto que lleva más tiempo esperando, se intercambian nombres y las funciones *dama* y *varon* de la pareja armada retornan. En ningún momento puede ocurrir que hay simultáneamente varones y damas en espera. El siguiente es un diagrama de threads que muestra un ejemplo de ejecución.



El diagrama muestra que la pareja de ana (D₁) es pedro (V₁) y que por lo tanto *dama* en D₁ retorna “pedro” y *varon* en V₁ retorna “ana”.

Programa las funciones *dama* y *varon* usando obligatoriamente la siguiente metodología:

- Debe usar **un solo mutex y una sola condición** de pthreads. No puede usar múltiples condiciones u otras herramientas de sincronización como por ejemplo semáforos.
- Las personas del mismo sexo deben conseguir pareja de baile por orden de llegada. Para ello use un distribuidor de números para las damas y otro distribuidor para los varones como se usó para el problema de los lectores/escritores visto en cátedra. **No puede usar colas fifo** para almacenar las damas o varones en espera.
- Necesitará usar variables globales para almacenar los distribuidores de tickets, los *displays*, el nombre de la dama o varón que espera pareja, etc.
- Si una persona le toca ser atendida, pero no encuentra pareja del sexo opuesto, colóquela en espera usando la condición, hasta que

aparezca su pareja. En ningún caso puede usar busy-waiting.

- Programe además la función *discoInit* para inicializar las variables globales que necesite y la función *discoDestroy* para liberar los recursos que solicitó. Si no las necesita, debe incluirlas de todas formas, pero pueden quedar vacías.

Instrucciones

Baje *t2.zip* de U-cursos y descomprímalo. El directorio *T2* contiene los archivos *test-disco.c*, *Makefile*, *disco.h* (con los encabezados requeridos) y otros archivos. Ud. debe programar en el archivo *disco.c* las funciones solicitadas. Defina otras funciones si las necesita.

Pruebe su tarea bajo Debian 12. Ejecute el comando *make* sin parámetros. Le mostrará las opciones que tiene para compilar su tarea. Estos son los requerimientos para aprobar su tarea:

- *make run* debe felicitarlo por aprobar este modo de ejecución.
- *make run-g* debe felicitarlo.
- *make run-san* debe felicitarlo y no reportar ningún incidente en el manejo de memoria.
- *make run-thr* debe felicitarlo y no reportar ningún datarace.

Cuando pruebe su tarea con *make run* en su computador asegúrese de que esté configurado en modo alto rendimiento y que no estén corriendo otros procesos intensivos en uso de CPU al mismo tiempo.

Invoque el comando *make zip* para ejecutar todos los tests y generar un archivo *disco.zip* que contiene *disco.c*, con su solución, y *resultados.txt*, con la salida de *make run*, *make run-g*, *make run-san* y *make run-thr*.

Entrega

Ud. solo debe entregar por medio de U-cursos el archivo *disco.zip* generado por *make zip*. Recuerde descargar el archivo que subió, descargar nuevamente los archivos adjuntos y volver a probar la tarea tal cual como la subió a U-cursos. Solo así estará seguro de no haber entregado archivos incorrectos. Se descuenta medio punto por día de atraso. No se consideran los días de receso, sábados, domingos o festivos.